

计算机网络实验报告

实验一：数据链路层滑动窗口协议的设计与实现

小组成员：张恒基、尹浩铭、林旭东

协议类型：搭载 ACK 的 Go-Back-N 协议

日期：2026 年 5 月

目录

1 实验内容和实验环境描述	3
1.1 实验任务与目标	3
1.2 实验环境	3
2 软件设计	4
2.1 数据结构	4
2.2 模块结构与算法流程	4
3 实验结果分析	5
3.1 性能测试记录与参数选择	5
3.2 理论推导与差距分析	6
4 研究和探索的问题	6
5 实验总结和心得体会	7
6 源程序文件	8

1 实验内容和实验环境描述

1.1 实验任务与目标

本实验的核心任务是利用数据链路层基本原理，设计并实现一个滑动窗口协议。实验要求在仿真环境下，完成有噪音信道下的两站点（A 站与 B 站）间的无差错全双工通信。通过本次实验，我们旨在深刻理解 CRC 校验技术以及滑动窗口的流量控制与差错恢复机理。

实验给定的信道模型如下：

- 信道带宽：8000 bps 全双工卫星信道。
- 传播时延：单向传播时延 270 ms。
- 信道误码率：默认 10^{-5} （即 1×10^{-5} ）。
- 物理层与网络层接口：网络层分组长度固定为 256 字节；数据链路层通过 `send_frame / recv_frame` 与物理层交互（物理层仿真实现 8000 bps 与传播时延等）。

本小组综合评估了实现难度与传输效率，最终选择实现的协议类型为：使用搭载 **ACK** 技术的 **Go-Back-N**（后退 N 帧）协议。

1.2 实验环境

- 硬件配置：Intel Core i7-12700H，16GB RAM。
- 操作系统与开发环境：Ubuntu 22.04 LTS，GCC 11.4.0（Linux 仿真环境）；Windows 下亦可使用 Visual Studio 打开 Lab1-Windows-VS2017/datalink.sln 编译同一套 src/ 源码。
- 工程结构：仓库根目录采用 Makefile 进行自动化构建。
- 启动方式：分别在两个终端执行 `./datalink -d3 A` 与 `./datalink -d3 B` 启动双端进程（调试输出可用 `-d3`）。

2 软件设计

2.1 数据结构

为了在网络层和物理层之间可靠地传递数据，我们定义了帧结构（与实现中 `struct frame / FRAME` 命名一致即可）。为便于组帧与 CRC 计算，字段顺序需与发送缓冲区布局一致：

变量名	类型	作用说明
<code>kind</code>	<code>unsigned char</code>	标识帧类型：如 <code>FRAME_DATA</code> 与 <code>FRAME_ACK</code>
<code>seq</code>	<code>unsigned char</code>	数据帧发送序号（取值范围由序号位数与窗口大小共同约束）
<code>ack</code>	<code>unsigned char</code>	捎带确认序号：表示该序号及以前的帧已被接收端正确接收（语义需在组内统一为「下一期望序号」或「累积 ACK」）
<code>data</code>	<code>unsigned char[256]</code>	载荷，与 <code>PKT_LEN</code> 一致，由 <code>get_packet()</code> 填充
<code>padding / CRC 区</code>	<code>unsigned int</code> 或附加 4 字节	CRC-32：实现中常在帧尾附加 4 字节校验域；若放在结构体末尾字段，须与 <code>put_frame</code> 长度计算一致

核心状态变量说明：

- `next_frame_to_send`：发送窗口上界，下一个待发新帧序号。
- `ack_expected`：发送窗口下界，已发送尚未被确认的最老帧序号。
- `frame_expected`：接收端按序期望的下一帧序号。
- `out_buf[]`：发送缓存，用于在 `DATA_TIMEOUT` 时从窗口下界起回退重传（GBN）。

2.2 模块结构与算法流程

本协议基于事件驱动模型，核心是一个运行在 `wait_for_event()` 上的状态机。

1. 初始化（`protocol_init`）：建立 TCP 连接、初始化日志与时间基准；链路层在准备好流量控制策略前可调用 `disable_network_layer()`，避免过早 `get_packet`。

2. 事件分发 (**switch (event)**) :

- **NETWORK_LAYER_READY**: 调用 `get_packet()` 取分组, 组帧、捎带 `ack`、计算 CRC32 并 `send_frame()`; 对未确认帧启动 `start_timer(seq, ...)`。
- **PHYSICAL_LAYER_READY**: 物理层发送队列低于约 50 字节; 若仍有待发帧应继续发送, 并维护「物理层可发」标志 (指导书 8.8: 本事件不会在未发送时重复投递)。
- **FRAME_RECEIVED**: `recv_frame()` 读入整帧; 先做 **CRC** 校验, 失败则丢弃。成功则根据 `ack` 滑动发送窗口并 `stop_timer`; 对按序到达的 DATA 调用 `put_packet()`; 根据策略 `start_ack_timer` / 立即发纯 ACK。
- **DATA_TIMEOUT**: GBN 从 `ack_expected` 起重传窗口内帧并重启相应定时器 (须保证重传路径上定时器被重新启动, 否则易出现「静默死锁」)。
- **ACK_TIMEOUT**: 无数据可捎带时发送纯 ACK 控制帧, 避免对接收方确认长期滞留。

3. 流量控制: 每次循环末尾根据「窗口是否已满」与物理层是否就绪, 在 `enable_network_layer()` 与 `disable_network_layer()` 之间切换。

3 实验结果分析

我们在不同模式与误码率下进行压力测试; 在实现完整 GBN 逻辑后, 程序能在默认误码与高误码场景下持续运行而不出现网络层「坏分组」中止 (指导书 8.13)。下表为 2026 年 5 月 15 日实测 600 秒运行末段 `lprintf` 统计数据。

3.1 性能测试记录与参数选择

序号	测试场景	命令选项	A 站利用率	B 站利用率
1	无误码数据传输	<code>-u -d0 -t 600</code>	39.36%	69.64%
2	默认平缓业务	<code>-d0 -t 600</code>	34.18%	61.81%
3	双端洪水 + 无误码	<code>-f -u -d0 -t 600</code>	72.32%	72.30%
4	双端洪水 + 默认误码	<code>-f -d0 -t 600</code>	63.69%	62.80%
5	洪水 + 高误码 (10^{-4})	<code>-f -b 1e-4 -d0 -t 600</code>	32.02%	31.24%

(注: 五个场景均自然 *Quit*, 日志中未出现 *bad packet / Abort*; 完整命令、端口与日志文件见 *docs/实验报告.typ* 附录。)

3.2 理论推导与差距分析

1. 窗口大小与定时器（定量草算）

设数据载荷 256 字节，若帧头与校验共 7 字节，则一帧长度 $L \approx 263$ 字节，合 2104 bit。

在 8000 bps 下，发送一帧所需传输时间 $T_x = \frac{2104}{8000} \approx 0.263$ s（约 263 ms）。单向传播时延

$T_p = 270$ ms。若 ACK 帧很短，其发送时间可近似忽略，则往返时间量级为：

$$\text{RTT} \approx 2T_p + T_x \approx 540 \text{ ms} + 263 \text{ ms} = 803 \text{ ms}$$

GBN 要保持「管道不空」，窗口内未确认数据在途时间应覆盖一次往返：粗略有 $W * T_x \geq$

$T_x + \text{RTT}$ ，代入得 $W \geq \frac{T_x + \text{RTT}}{T_x} \approx \frac{810}{263} \approx 3.08$ 。本组按字节 BDP 折中取 WINDOW_SIZE=3。

序号字段为 1 字节，最终使用完整 MAX_SEQ=255（0-255）空间，避免误码重传时旧帧在物理层队列中滞留、序号过快回绕后被接收端误收。

重传定时器 DATA_TIMEOUT_MS=600 略大于传播往返 540 ms；库函数 start_timer 还会自动叠加 phl_sq_len 折算出的本地物理层排队发送时间，减少帧尚未离站就被误判超时的情况。

2. 利用率上界与实测对比

无误码且载荷占主导时，线路利用率上界近似为数据字节占帧长比例： $\frac{256}{263} \approx 97.34\%$ 。表

中情景 3 的 72.3% 低于理想上界，主要受 WINDOW_SIZE=3、ACK 等待、事件调度以及物理层排队影响；但两端数值接近，说明全双工洪水下窗口能稳定滑动。

在误码 10^{-4} 的洪水情景（表 5），GBN 的「一处出错、退回重传」会丢弃大量本已正确到达的后续帧，利用率下降到约 31%-32%。这与理论预期一致，也说明在恶劣信道上选择重传（SR）往往更划算。

4 研究和探索的问题

问题：CRC 能否支撑「无差错传输」的客户预期？（尹浩铭）

CRC-32（IEEE 802.3 标准，生成多项式 $G(x) = x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$ ）在实验中的使用方式为：发送端在帧末尾追加 4 字节 CRC 校验值，接收端对整帧（含 CRC 字段）计算 $\text{crc32}(\dots)$ ，若结果不为 0 则判定帧已损坏。

漏检概率的理论估计：CRC-32 的漏检概率 $\approx 2^{-32} \approx 2.33 \times 10^{-10}$ （假设错误模式均匀随机）。设链路利用率为 η ，信道速率 $B = 8000 \text{ bps}$ ，帧长 $L_{\text{bit}} = 2104$ ，则每秒发送帧数约 $\eta \frac{B}{L_{\text{bit}}}$ 。取 $\eta = 0.5$ 得每日约 1.64×10^5 帧，漏检一帧所需平均天数约 72 年——在统计上可忽略。

默认 $P_b = 10^{-5}$ 时，一帧至少出现 1 bit 错误概率约 2.08%，即每 48 帧就有 1 帧被 CRC 检出丢弃；漏检概率比可检出错误低约 8 个数量级。CRC-32 在本实验场景中足够可靠。

相关阅读：start_timer 与 start_ack_timer 的设计差异见 src/datalink_recv.c、src/datalink.c 及 docs/实验报告.typ 中的探索二论述。

问题：start_timer 与 start_ack_timer 的设计差异（尹浩铭）

根据本仓库 src/protocol.c 中的实现：

- start_timer(nr, ms) 到期时刻含 `phl_sq_len() × 8000 / CHAN_BPS` 排队项（字节数折算发送时间），避免帧还在队列中未上线路就超时导致的虚假重传。
- start_ack_timer(ms) 仅当定时器未运行时启动（`if (timer[ACK_TIMER_ID] == 0)`），不因每次 FRAME_RECEIVED 都重置。保证在连续接收数据帧时，ACK 定时器不会被无限推后，纯 ACK 不会无限拖延。
- 若二者的语义互换或混淆（如用 start_timer 驱动纯 ACK），排队项会大幅延迟 ACK 发送，在洪水场景下可能造成对端窗口停滞。

5 实验总结和心得体会

- 实际上机时间：约 15 小时（含阅读指导书、编码、联调与写报告）。
- 团队协作与分工：
 - 张恒基：状态机主循环与窗口边界条件；处理 start_timer / stop_timer 与物理层就绪的协同。
 - 尹浩铭：帧结构与 CRC 集成；负责 datalink_recv.c 接收侧全部逻辑（CRC 校验、重复/失序检测、按序交付 put_packet、纯 ACK 组帧与携带 ACK 维护）；排查结构

体布局、长度与 `recv_frame` 缓冲区导致的内存问题；撰写 11.4 探索一（CRC 漏检分析）与探索二（定时器设计差异）。

- 林旭东：长时压测与数据记录；维护 `Makefile` / `CI` 脚本；整理日志并与理论曲线对照。
- 协议死锁调试经历：早期曾出现运行数分钟后双方利用率归零：分析 `datalink-A.log` 发现 `DATA_TIMEOUT` 重传路径未重新 `start_timer`，若重传帧再次因误码被丢弃，发送端将不再超时唤醒，窗口卡死。补齐重传后定时器重启逻辑后，长时间高误码洪水测试恢复稳定。
- 序号空间调试经历：早期 `MAX_SEQ=7` 版本在默认误码长跑中曾出现网络层坏分组。原因是旧重传帧可能在物理层队列中滞留到接收端序号绕回后被误收。最终将 1 字节序号空间完整用于编号，改为 `MAX_SEQ=255` / `NR_BUFS=256`，窗口仍为 3；复跑五个表 3 场景各 600 秒均无坏分组。
- 总结：通过实现搭载 `ACK` 的 `GBN`，我们把教材中的流量控制与差错控制落实为可观测的日志与时间线，体会到协议软件中「边界条件」与「定时器语义」对正确性的决定性影响。

6 源程序文件

本仓库核心自研文件为 `src/datalink.c`、`src/datalink_recv.c`、`include/datalink.h`；教师仿真库位于 `src/protocol.c` 等。以下为与报告描述一致的帧结构与接收分支示意：

```
/* include/datalink.h 帧结构定义 */  
  
#define FRAME_HDR_LEN 3  
  
#define MAX_SEQ 255  
  
#define NR_BUFS 256  
  
#define MAX_FRAME_BYTES (FRAME_HDR_LEN + PKT_LEN + 4)  
  
struct frame {  
    unsigned char kind;  
    unsigned char seq;  
    unsigned char ack;
```



```
    unsigned char data[PKT_LEN];  
};  
  
/* src/datalink_rcv.c 接收处理核心 (尹浩铭) */  
int validate_and_process_frame(unsigned char *frame, int len,  
    unsigned int *ack_seq,  
    unsigned char *data_out, int *data_len)  
{  
    if (len < FRAME_HDR_LEN + 4) return -1;  
    if (crc32(frame, (unsigned)len) != 0) return -1;  
    if (f->kind == FRAME_ACK) { *ack_seq = f->ack; return 0; }  
    if (f->seq != frame_expected) return 2;  
    memcpy(data_out, f->data, PKT_LEN);  
    *data_len = PKT_LEN;  
    frame_expected = (frame_expected + 1) % (MAX_SEQ + 1);  
    return 1;  
}
```